

A Finite-Field and Product-Construction Approach to Strength-Two Orthogonal Array Generation for Software Testing

Siddharth Narayan Mishra

Department of Computer Science and Engineering

National Institute of Technology, Rourkela

Roll Number: 123CS0189

123cs0189@nitrkl.ac.in

August 17, 2026

Abstract

Orthogonal arrays (OAs) provide a classical combinatorial foundation for pairwise (strength-two) software testing, guaranteeing that every ordered pair of values across any two factors is exercised the same number of times. This paper presents the design, implementation, and empirical verification of a general-purpose strength-two orthogonal array generator. Rather than proposing a new construction, the contribution lies in a complete, automated pipeline that (i) constructs prime-power-level arrays using the classical finite-field/projective-geometry method, (ii) extends to arbitrary composite level counts via the direct product construction, and (iii) independently verifies the pairwise-balance property of every generated array. We give a rigorous, self-contained proof that the product construction preserves strength-two orthogonality – a step that is often stated without proof in applied treatments – and we analyze the time and space complexity of the resulting algorithm. We further report empirical results across eighteen (k, s) configurations, in which every generated array was verified correct and array sizes matched the theoretical prediction $N = \lambda s^2$ exactly. We situate this implementation within the broader literature on finite-field constructions of combinatorial arrays and on combinatorial interaction testing (CIT), and we discuss the scope and limitations of orthogonal arrays relative to the more flexible covering-array model used in modern industrial test generation tools.

Keywords: orthogonal arrays, finite fields, combinatorial testing, pairwise testing, software testing, Galois fields, covering arrays

1 Introduction

Exhaustive testing of a software system with k configurable factors, each taking s possible values, requires s^k test cases – a number that becomes computationally and practically infeasible even for modest k and s . Combinatorial testing addresses this explosion by selecting a much smaller subset of test cases that still guarantees a specified strength of coverage over all factor interactions. The simplest and most widely used instance of this idea is *pairwise* (strength-two) testing, in which every ordered pair of values across every pair of factors is required to appear in at least one test case. Empirical studies of field failures have repeatedly found that a large fraction of software defects are triggered by interactions between at most two parameters, which explains the continued practical relevance of pairwise coverage as a baseline testing strategy [8].

Orthogonal arrays (OAs) are the classical, statistically motivated combinatorial object underlying pairwise testing: an $N \times k$ array over an s -symbol alphabet is a strength-two OA if every ordered pair of symbols occurs the same number of times, λ , in every pair of columns. Because this balance property is exact rather than merely a covering guarantee, OAs give test suites with additional statistical structure – useful, for example, when test outcomes are analyzed for factor effects rather than simply checked for pass/fail – at the cost of existing only for restricted combinations of (N, k, s) .

The theory of OA construction is well developed, resting on finite-field and finite-geometry methods that go back to Bose and Bush [1] and are comprehensively surveyed by Hedayat, Sloane, and Stufken [2]. The present work does not propose a new construction. Its contribution is implementation: given an arbitrary number of factors $k \geq 2$ and a common number of levels $s \geq 2$, the system automatically (a) determines suitable construction parameters, (b) builds the array using the finite-field construction when s is a prime power and the direct product construction otherwise, and (c) independently verifies the pairwise-balance property by exhaustive counting. We additionally provide a complete proof that the product construction preserves strength-two orthogonality under repeated composition (Lemma 1), a fact that is used constantly in the literature but rarely proved in full for the general m -factor composite case, and we report the complexity and empirical runtime behavior of the resulting algorithm.

Contributions. Concretely, this paper contributes:

1. A complete, parameter-general implementation combining the finite-field construction (Section 4) with the direct product construction (Section 5) to handle any $s \geq 2$, not only prime powers.
2. A rigorous proof, by explicit bijective decomposition, that the direct product of strength-two orthogonal arrays is again a strength-two orthogonal array under iterated composition (Lemma 1 and Corollary 1).
3. A time and space complexity analysis of the full generation and verification pipeline (Section 6.1).
4. An empirical evaluation over eighteen (k, s) configurations spanning prime, prime-power, and composite level counts, with all arrays independently verified (Section 7).

2 Related Work

2.1 Classical orthogonal array theory

The formal study of orthogonal arrays of strength two and three originates with Bose and Bush [1], who established the finite-field and finite-geometry constructions that remain the standard approach for prime-power level counts. Hedayat, Sloane, and Stufken [2] give the definitive modern treatment, unifying constructions based on finite fields, difference schemes, Hadamard matrices, and Latin squares, and cataloguing the known existence results and open problems for asymmetric (mixed-level) arrays. Bayrak and Alhan [3] revisit the finite-geometric construction used in this paper, framing strength-two OAs as arising from the incidence structure of a projective space, which is the same viewpoint adopted in Section 4.

2.2 Finite-field constructions of combinatorial arrays

More recent work has systematized finite-field methods across a wider family of combinatorial arrays. Moura, Mullen, and Panario [4] survey finite-field constructions not only of orthogonal arrays but of

covering arrays, ordered orthogonal arrays, and related objects, and discuss how variable-strength and mixed-strength requirements can be layered on top of the basic finite-field construction used here. Their survey situates the classical construction of Section 4 – selecting non-proportional vectors in $\text{GF}(q)^d$ and evaluating linear functionals – as one instance of a broader design pattern in algebraic combinatorics. The finite-field arithmetic itself, including polynomial representations of $\text{GF}(p^r)$ for non-prime q , follows the standard treatment of Lidl and Niederreiter [5].

2.3 Covering arrays and combinatorial interaction testing

Within software engineering, orthogonal arrays are one of two dominant combinatorial models used for interaction testing, the other being covering arrays (CAs). A CA relaxes the OA requirement from *exact* pairwise balance to *at least one* occurrence of every pair, which in general permits substantially smaller test suites and, critically, removes the restriction that all factors share the same number of levels or that (N, k, s) satisfy the algebraic divisibility conditions an OA requires. This trade-off is discussed directly in NIST’s comparative treatment of factorial designs, covering arrays, and combinatorial testing [6], which notes that a CA meeting a given strength typically needs fewer rows than the corresponding OA and can accommodate unequal level counts and forbidden factor combinations – properties an OA-based generator such as ours does not provide. Lei and Tai’s In-Parameter-Order (IPOG) algorithm [7] was among the first greedy strategies for constructing pairwise covering arrays directly, without going through an algebraic OA construction, and remains the basis of widely used tools such as ACTS. Kuhn, Kacker, and Lei’s NIST report on practical combinatorial testing [8] provides the empirical motivation, summarized in Section 1, for why pairwise (and low-strength) interaction coverage is effective at detecting real software faults. More recent systems work, such as Ukai et al.’s combinatorial-join technique for accelerating industrial-scale covering array generation [9], illustrates that CA generation performance is an active engineering concern in its own right, in contrast to the comparatively rigid but exact algebraic route taken in this paper.

2.4 Positioning of the present work

The system described here sits deliberately on the algebraic, OA side of this landscape rather than the heuristic CA side. This choice trades the flexibility that CA generators such as IPOG offer – arbitrary strengths, unequal level counts, constraint handling – for two properties that are harder to get from greedy or search-based CA construction: an exact, closed-form guarantee on row count ($N = \lambda s^2$) derived directly from finite-field theory, and a construction whose correctness can be proved rather than only empirically tested. Section 8 returns to this trade-off explicitly.

3 Preliminaries

Definition 1 (Orthogonal array). *An orthogonal array $OA_\lambda(N, k, s, 2)$ is an $N \times k$ array A with entries from a symbol set of size s such that, for every pair of distinct columns $i, j \in \{1, \dots, k\}$ and every ordered pair of symbols (a, b) , the number of rows r with $A[r, i] = a$ and $A[r, j] = b$ is exactly λ , independent of (a, b) and of the choice of i, j . The array has strength two and index λ , and necessarily $N = \lambda s^2$.*

Definition 2 (Finite field $\text{GF}(q)$). *For a prime power $q = p^r$, $\text{GF}(q)$ denotes the finite field with q elements. For $r = 1$, $\text{GF}(q) \cong \mathbb{Z}_q$ under addition and multiplication modulo q . For $r > 1$, $\text{GF}(q)$ is represented as the quotient ring $\mathbb{Z}_p[x]/(f(x))$ for a degree- r polynomial f irreducible over $\mathbb{Z}_p$.*

Throughout, k denotes the number of factors (columns) and s the common number of levels (symbols) per factor.

4 Construction: Prime-Power Number of Levels

When $s = q$ is a prime power, the array is built directly from the linear structure of the vector space $\text{GF}(q)^d$.

4.1 Choice of dimension

Let d be the smallest positive integer satisfying

$$\frac{q^d - 1}{q - 1} \geq k. \quad (1)$$

The left-hand side counts the one-dimensional subspaces (equivalently, the points of the projective space $PG(d - 1, q)$) of $\text{GF}(q)^d$, so condition (1) guarantees enough distinct projective directions to assign one to each of the k factors.

4.2 Choice of representative vectors and array construction

Select k vectors $v_1, \dots, v_k \in \text{GF}(q)^d$ no two of which are scalar multiples of one another (equivalently, one representative per required one-dimensional subspace). For every $x \in \text{GF}(q)^d$, create one row of the array and set

$$A[x, j] = x \cdot v_j, \quad j = 1, \dots, k, \quad (2)$$

where $x \cdot v_j$ is the standard dot product evaluated in $\text{GF}(q)$. The resulting array has $N = q^d$ rows and k columns over an $s = q$ -symbol alphabet. Algorithm 1 summarizes the procedure.

Algorithm 1 Prime-power OA construction

Require: Number of factors k ; prime power q

- 1: Find the smallest d such that $(q^d - 1)/(q - 1) \geq k$
 - 2: Construct $\text{GF}(q)$ (modular arithmetic if q prime; polynomial arithmetic modulo an irreducible degree- r polynomial if $q = p^r$, $r > 1$)
 - 3: Choose k non-proportional vectors $v_1, \dots, v_k \in \text{GF}(q)^d$
 - 4: **for** all $x \in \text{GF}(q)^d$ **do**
 - 5: **for** $j \leftarrow 1$ to k **do**
 - 6: $A[x, j] \leftarrow x \cdot v_j$
 - 7: **end for**
 - 8: **end for**
 - 9: **return** A
-

Theorem 1. *For a prime power q , the array produced by Algorithm 1 is a strength-two orthogonal array $OA_{q^{d-2}}(q^d, k, q, 2)$.*

Proof. Consider two distinct columns i, j , corresponding to vectors v_i, v_j . Since v_i, v_j are not scalar multiples of one another, they are linearly independent over $\text{GF}(q)$. Fix any target pair $(a, b) \in \text{GF}(q)^2$. The system

$$x \cdot v_i = a, \quad x \cdot v_j = b$$

is a system of two linearly independent linear equations in the d unknowns comprising $x \in \text{GF}(q)^d$. Over a field, such a system has exactly q^{d-2} solutions, obtained by fixing the two coordinates constrained by the (linearly independent) equations and letting the remaining $d - 2$ coordinates range freely over $\text{GF}(q)$. Hence every ordered pair (a, b) occurs in columns i, j exactly q^{d-2} times, independent of (a, b) , i , and j . By Definition 1, this is precisely strength-two orthogonality with index $\lambda = q^{d-2}$. $\square$

5 Construction: Composite Number of Levels

Algorithm 1 only applies when s is itself a prime power. For general s , we factor s into pairwise coprime prime powers, build a component array for each factor, and combine them.

5.1 Direct product of two orthogonal arrays

Definition 3 (Direct product). *Let A be an $OA_{\lambda_A}(N_A, k, s_A, 2)$ over symbols $\{0, \dots, s_A - 1\}$ and B an $OA_{\lambda_B}(N_B, k, s_B, 2)$ over symbols $\{0, \dots, s_B - 1\}$, both with the same number of columns k . The direct product $C = A \otimes B$ is the $N_A N_B \times k$ array whose rows are indexed by pairs (r_A, r_B) with r_A a row of A and r_B a row of B , and whose entries are*

$$C[(r_A, r_B), j] = A[r_A, j] \cdot s_B + B[r_B, j], \quad j = 1, \dots, k, \quad (3)$$

taking values in $\{0, \dots, s_A s_B - 1\}$.

Lemma 1. *If A is $OA_{\lambda_A}(N_A, k, s_A, 2)$ and B is $OA_{\lambda_B}(N_B, k, s_B, 2)$, then $C = A \otimes B$ is $OA_{\lambda_A \lambda_B}(N_A N_B, k, s_A s_B, 2)$*

Proof. By construction, C has $N_A N_B$ rows and k columns. We must show the pairwise-balance property of Definition 1 holds over the $s_A s_B$ -symbol alphabet with a constant index.

The encoding map $\phi : \{0, \dots, s_A - 1\} \times \{0, \dots, s_B - 1\} \rightarrow \{0, \dots, s_A s_B - 1\}$, $\phi(a, b) = as_B + b$, is precisely the base- s_B positional encoding of the pair (a, b) and is therefore a bijection: every symbol $u \in \{0, \dots, s_A s_B - 1\}$ has a unique decomposition $u = \phi(a_u, b_u)$ with $a_u = \lfloor u/s_B \rfloor$ and $b_u = u \bmod s_B$.

Fix two distinct columns $i, j \in \{1, \dots, k\}$ and a target ordered pair (u, v) of symbols of C , with unique decompositions $u = \phi(a_u, b_u)$ and $v = \phi(a_v, b_v)$. A row (r_A, r_B) of C satisfies $C[(r_A, r_B), i] = u$ and $C[(r_A, r_B), j] = v$ if and only if, by injectivity of ϕ applied coordinatewise,

$$A[r_A, i] = a_u, \quad A[r_A, j] = a_v \quad \text{and} \quad B[r_B, i] = b_u, \quad B[r_B, j] = b_v.$$

These two conditions are independent, one constraining only r_A and the other only r_B . Since A is orthogonal with index λ_A , exactly λ_A choices of r_A satisfy the first condition; since B is orthogonal with index λ_B , exactly λ_B choices of r_B satisfy the second. By the product rule, the number of rows (r_A, r_B) satisfying both is $\lambda_A \lambda_B$, independent of (u, v) , i , and j . Hence C is a strength-two orthogonal array with index $\lambda_A \lambda_B$. $\square$

Corollary 1 (Iterated product). *Let $s = q_1 q_2 \cdots q_m$ be a factorization of s into pairwise coprime prime powers, and let $A_1, \dots, A_m$ be strength-two orthogonal arrays with k columns and $q_1, \dots, q_m$ levels respectively, produced by Algorithm 1. Then $A = A_1 \otimes A_2 \otimes \cdots \otimes A_m$ (associated left to right) is a strength-two orthogonal array with k columns, s levels, and $N = \prod_{t=1}^m N_t = \prod_{t=1}^m q_t^{d_t}$ rows.*

Proof. By induction on m using Lemma 1: the base case $m = 1$ is Theorem 1. For the inductive step, assume $A_1 \otimes \cdots \otimes A_{m-1}$ is a valid strength-two OA with $\prod_{t < m} q_t$ levels; applying Lemma 1 once more with $B = A_m$ preserves the property and multiplies the level count by q_m and the row count by N_m . $\square$

Because the q_t are pairwise coprime, distinct symbols $u \in \{0, \dots, s-1\}$ obtained from the iterated encoding correspond to distinct residue tuples modulo each q_t by the Chinese Remainder Theorem, so the final relabelling to $\{1, \dots, s\}$ in Algorithm 2 is a straightforward bijective renaming and introduces no additional collisions.

Algorithm 2 General orthogonal array generation

Require: Integers $k \geq 2$ and $s \geq 2$

- 1: Factor $s = q_1 q_2 \cdots q_m$ into pairwise coprime prime powers
 - 2: **for** $t \leftarrow 1$ to m **do**
 - 3: $A_t \leftarrow \text{PRIMEPOWEROA}(k, q_t)$ ▷ Algorithm 1
 - 4: **end for**
 - 5: $A \leftarrow A_1$
 - 6: **for** $t \leftarrow 2$ to m **do**
 - 7: $A \leftarrow \text{DIRECTPRODUCT}(A, A_t)$ ▷ Equation (3)
 - 8: **end for**
 - 9: Relabel symbols as $1, \dots, s$
 - 10: **return** A
-

6 Implementation

The system is implemented in C++ and consists of four principal components, corresponding directly to Algorithms 1 and 2:

1. **Integer factorization.** Trial division factors s into prime powers in $O(\sqrt{s})$ time.
2. **Finite-field arithmetic.** Prime q uses arithmetic modulo q ; prime-power $q = p^r$ with $r > 1$ uses polynomial arithmetic modulo a fixed irreducible degree- r polynomial over $\mathbb{Z}_p$.
3. **Canonical subspace representatives.** Vectors $v_1, \dots, v_k \in \text{GF}(q)^d$ are generated so that no two are scalar multiples, by fixing the first nonzero coordinate of each representative (the standard canonical form for points of $PG(d-1, q)$).
4. **Direct product composition.** Component arrays are combined pairwise according to Equation (3) until all prime-power factors of s have been incorporated.

6.1 Complexity analysis

Let $q_{\max}$ denote the largest prime-power factor of s and $d = O(\log_{q_{\min}} k)$ the dimension required by condition (1) for the corresponding component (since $(q^d - 1)/(q - 1) \geq k$ forces $d = O(\log_q k)$ for fixed $q \geq 2$).

- *Factorization:* $O(\sqrt{s})$.
- *Single prime-power component (Algorithm 1):* enumerating $\text{GF}(q)^d$ produces q^d rows, and each row requires k dot products of length d , giving $O(k d q^d)$ time and $O(k q^d)$ space for that component.
- *Direct product (Algorithm 2):* combining two component arrays with N_A and N_B rows takes $O(k N_A N_B)$ time and space to materialize the $N_A N_B$ -row result, since every pair of component rows is combined once across all k columns.

- *Overall generation:* for $s = q_1 \cdots q_m$ with resulting total row count $N = \prod_t N_t$, the dominant cost is the final materialization of the full $N \times k$ array, giving $O(kN)$ time and space overall, since each intermediate product is no larger than the final array.
- *Verification:* for each of the $\binom{k}{2}$ column pairs, counting all N rows against an $s \times s$ frequency table costs $O(N + s^2)$, giving $O(k^2(N + s^2))$ overall.

Because $N = \lambda s^2$ grows multiplicatively with the number of distinct prime-power factors of s and exponentially with d (hence, for fixed q , roughly linearly-to-polynomially with k via $d = O(\log_q k)$), the generator is efficient for the moderate (k, s) regime typical of software test-parameter spaces, but N can grow quickly for level counts s with many distinct prime-power factors, since each additional coprime factor multiplies N by that component’s own row count.

7 Experimental Verification

To validate both the correctness and the practical performance of the implementation, we ran the generator over eighteen (k, s) configurations chosen to exercise prime, prime-power, and composite level counts, and to span a range of factor counts. For every configuration, we recorded the number of factors k , the number of levels s , the number of generated rows N , the resulting index λ , the wall-clock generation time, and the outcome of the independent verification procedure of Section 4–5 (exhaustive counting of all s^2 ordered symbol pairs across every column pair). All experiments were run on a single core; timings are averaged over process wall-clock time including I/O.

Table 1: Experimental results across eighteen (k, s) configurations. N and λ are the array size and index predicted by $N = \lambda s^2$; every configuration passed independent pairwise-balance verification.

k	s	N (rows)	λ	Time (ms)	Verified
3	2	4	1	4.59	PASS
4	2	8	2	3.51	PASS
5	2	8	2	3.40	PASS
7	2	8	2	3.36	PASS
10	2	16	4	2.21	PASS
4	3	9	1	3.42	PASS
5	3	27	3	3.81	PASS
6	3	27	3	3.52	PASS
4	4	16	1	3.44	PASS
5	5	25	1	3.33	PASS
3	6	36	1	6.73	PASS
4	6	72	2	3.70	PASS
5	7	49	1	2.34	PASS
4	8	64	1	3.31	PASS
6	4	64	4	3.51	PASS
6	9	81	1	3.62	PASS
3	10	100	1	3.26	PASS
4	12	144	1	2.16	PASS

7.1 Discussion of results

Three observations follow from Table 1. First, in every case the measured row count N matches the closed-form prediction $N = \lambda s^2$ exactly, and independent verification passed in all eighteen configurations, corroborating Theorem 1 and Lemma 1 empirically. Second, composite level counts confirm the direct product behavior of Corollary 1: for example, $s = 6 = 2 \cdot 3$ with $k = 4$ yields $N = 72 = 8 \times 9$, the product of the component row counts for $(k=4, q=2)$ and $(k=4, q=3)$; similarly $s = 12 = 4 \cdot 3$ yields $N = 144 = 16 \times 9$. Third, generation time remains under 7 ms for every configuration tested, dominated by process startup overhead rather than by the array construction itself, since even the largest instance tested ($N = 144$ rows) is small relative to the polynomial-time bounds of Section 6.1; we would expect runtime to become construction-dominated only for substantially larger k or s with many distinct prime-power factors.

8 Discussion: Scope and Limitations

The implementation assumes a common number of levels s across all k factors and generates only strength-two arrays; it does not address mixed-level (asymmetric) designs or strengths $t > 2$, both of which are covered in the general theory of Hedayat, Sloane, and Stufken [2] but require substantially different constructions. Nor does the finite-field/product route minimize N : as Section 2 discusses, a covering array satisfying the same pairwise-coverage requirement can typically be built with fewer rows, particularly when k is large relative to s or when factors have unequal level counts [6]. Algorithms such as IPOG [7] trade the exact algebraic guarantees of Theorem 1 for this row-count efficiency and for the ability to handle constraints between factor values, which the present system does not support.

The value of the OA-based approach used here is therefore not row-count optimality but two properties that are difficult to obtain from search-based CA construction: a provable, closed-form bound on array size before generation begins, and a construction whose pairwise-balance property follows from a short algebraic proof (Theorem 1, Lemma 1) rather than from empirical search alone. This makes the system best suited to settings where either the exact-balance property of an OA is itself desired – for example, when downstream statistical analysis of test outcomes assumes balanced designs – or where k and s are small enough that OA row counts remain competitive with CA row counts, which the experiments of Section 7 suggest holds for the parameter ranges typical of unit- and component-level pairwise test suites.

9 Conclusion and Future Work

We presented a complete implementation of a strength-two orthogonal array generator combining the classical finite-field construction with the direct product construction, together with a rigorous proof (Lemma 1, Corollary 1) that the product construction preserves pairwise balance under iterated composition – a result stated but not proved in the applied literature we surveyed. We analyzed the time and space complexity of the resulting pipeline and validated it empirically across eighteen configurations spanning prime, prime-power, and composite level counts, with all generated arrays independently verified correct and all row counts matching the theoretical prediction exactly.

Natural extensions, informed by the related-work survey of Section 2, include: (i) support for mixed-level (asymmetric) orthogonal arrays, following the finite-geometric methods surveyed by Moura et al. [4]; (ii) a hybrid mode that falls back to a covering-array heuristic such as IPOG [7] when the algebraic OA row count exceeds a configurable budget, combining the exactness of the

present approach with the row-count efficiency of CA methods when $k \gg s$; and (iii) extension to strengths $t > 2$ using higher-dimensional analogues of Theorem 1, for which the finite-field machinery generalizes directly but the corresponding product lemma would need to be re-derived for t -wise rather than pairwise balance.

References

- [1] R. C. Bose and K. A. Bush, “Orthogonal Arrays of Strength Two and Three,” *The Annals of Mathematical Statistics*, vol. 23, no. 4, pp. 508–524, 1952.
- [2] A. S. Hedayat, N. J. A. Sloane, and J. Stufken, *Orthogonal Arrays: Theory and Applications*. New York, NY, USA: Springer, 1999.
- [3] H. Bayrak and A. Alhan, “On the Construction of Orthogonal Arrays,” *Hacettepe Journal of Mathematics and Statistics*, vol. 31, pp. 45–51, 2002.
- [4] L. Moura, G. L. Mullen, and D. Panario, “Finite Field Constructions of Combinatorial Arrays,” *Designs, Codes and Cryptography*, vol. 78, pp. 197–219, 2016.
- [5] R. Lidl and H. Niederreiter, *Finite Fields*, 2nd ed. Cambridge, UK: Cambridge University Press, 1997.
- [6] R. N. Kacker, D. R. Kuhn, Y. Lei, and R. M. Kuhn, “Factorial Experiments, Covering Arrays, and Combinatorial Testing,” National Institute of Standards and Technology, Gaithersburg, MD, USA, Tech. Rep., 2021.
- [7] Y. Lei and K.-C. Tai, “In-Parameter-Order: A Test Generation Strategy for Pairwise Testing,” in *Proc. 3rd IEEE Int. High-Assurance Systems Engineering Symp.*, Washington, DC, USA, 1998, pp. 254–261.
- [8] D. R. Kuhn, R. N. Kacker, and Y. Lei, “Practical Combinatorial Testing,” NIST Special Publication 800-142, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2010.
- [9] H. Ukai, X. Qu, H. Washizaki, and Y. Fukazawa, “Accelerating Covering Array Generation by Combinatorial Join for Industry Scale Software Testing,” *PeerJ Computer Science*, vol. 8, e720, 2022.